

5 Software Reuse

Topics

- Introduction to Software Reuse
- Benefits and Challenges of Software Reuse
- Key Software Reuse Strategies:
 - Application Framework
 - Software Product Lines
 - Commercial Off-The-Shelf

Introduction to Software Reuse

- Why software reuse matter?
- Companies like Google and Netflix save millions by reusing code. Today, we will learn how to do the same

谷歌和Netflix等公司通过代码复用节省了数百万成本。今天，我们将学习如何实现同样的效果。
- Definition: Software reuse is the practice of using existing software components (code, designs, etc.) to build new system

定义：软件复用是指利用现有软件组件（代码、设计等）构建新系统的实践。
- Real-world impact:

现实世界的影响

 - Faster development

更快的开发

 - TensorFlow powers multiple AI tools at Google

TensorFlow为谷歌的多种人工智能工具提供支持。
 - Google Launches TensorFlow Enterprise at No Extra Cost for Cloud Customers

谷歌为企业云客户推出TensorFlow Enterprise服务且不额外收费
 - Fewer bugs

更少的bug

 - React's reusable components reduce errors across Meta's platforms

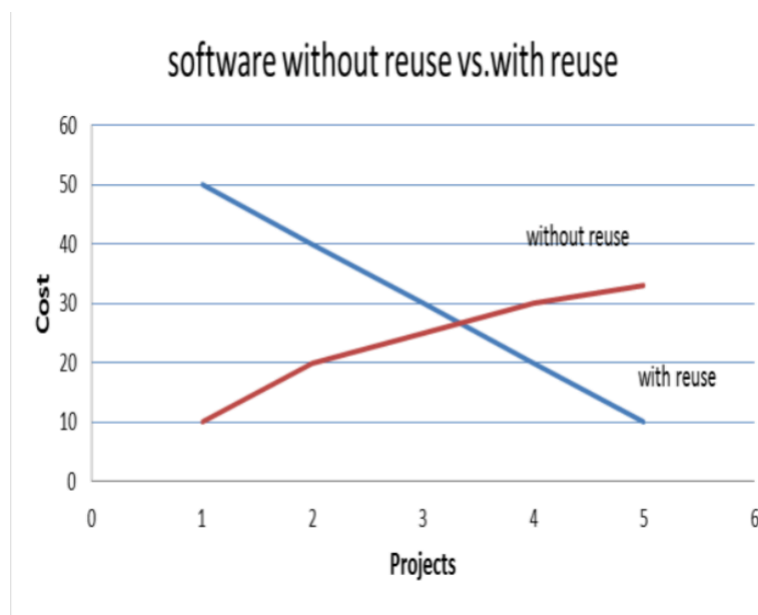
React的可复用组件降低了Meta各平台中的错误率。
 - Facebook, Instagram, and WhatsApp
- Why it's exciting: Reuse isn't just efficient – It's key skill in modern software engineering

为何令人振奋：代码复用不仅是效率的提升——它更是现代软件工程中的核心技能。

Benefit of Software Reuse 软件复用的好处

saves time and money?

- Key benefit:
 - Cost savings: Reuse reduces development and maintenance cost
成本节约：重复利用可降低开发和维护成本
 - Faster delivery: Get products to market quicker
更快的交付：缩短产品推向市场的时间（Time-to-market）
 - Higher quality: Tested components mean fewer bugs
提高质量：经过前人多次测试的组件比新写的代码更可靠
 - Specialist knowledge: Leverage experts' work without reinventing the wheel
利用专家知识：直接利用该领域专家的成果，避免在不熟悉的领域犯错。
- Example: Spotify reuses UI compnents across its web and mobile apps for consistency and speed.
例如，Spotify在其网页和移动应用中复用UI组件，以保持一致性并提升速度。



Challenges of Software Reuse 代码复用的劣势

reuse isn't always easy?

- Key challenges:
 - Maintenance costs: Reuse code may become outdated or incompatible
维护成本：复用代码可能过时或不兼容
 - Tool support: Some tools don't integrate well with reuse components
工具支持：部分工具与可复用组件集成效果不佳
 - Not-invented-here syndrome: Developers may resist using others' code
非我发明症候群：开发者可能倾向于拒绝使用他人编写的代码。
 - Finding and Adapting components: It can be hard to locate and modify the right components
寻找与适配组件：定位并修改合适的组件可能颇具挑战
- Can you think of a time when reusing something didn't work out?

Challenges of Software Reuse

- Example:
 - **Ariane 5 Flight 501 (1996):** The Ariane 5 rocket exploded 40 seconds after launch due to a software error. The software
 - **Knight Capital Group (2012):** Knight Capital Group deployed new trading software that reused old code from a previous system. The old code was not properly deactivated, leading to a series of erroneous trades.
 - **Outcome:** The company lost \$440 million in just 45 minutes and nearly went bankrupt. This incident highlighted the risks of reusing legacy code without proper integration and testing.
 - **Log4j Vulnerability (2021):** The Log4j library, a widely reused open-source logging tool, had a critical vulnerability (CVE-2021-44228) that allowed remote code execution. Many organizations had reused Log4j in their systems without fully understanding its dependencies

Core Reuse Techniques 核心的复用技巧

- Key approaches:
 - **Application framework:** Pre-built structures (e.g. Django, Spring) that you extend.
应用框架：可扩展的预构建结构（如 Django、Spring）
 - **Software produce lines (SPL):** Families of similar products with shared components (e.g. Salesforce CRM variants)
软件产品线（SPL）：具有共享组件的相似产品系列（例如Salesforce CRM变体）
 - **COTS (Commercial Off-The-Shelf):** Ready-made software adapted for specific needs (e.g. WordPress for websites)
COTS（商用现成品或技术）：为特定需求而调整的现成软件（例如用于网站的WordPress）
 - Other: Component-based, open-source libraries, design patterns, and more
其他：基于组件的开源库、设计模式等

Key Software Reuse Strategies 关键软件复用策略

Application Framework 应用框架

- Application framework: Build on a solid foundation
应用框架：构建于坚实基础之上
- Definition: A generic structure (e.g. a set of classes) that you extend to create specific applications
定义：一种通用结构（例如一组类），通过扩展该结构来创建特定应用程序。
- Examples:
 - Web framework: Django (Python), Spring (Java)
Web框架：Django（Python），Spring（Java）
 - Mobile framework: Flutter (cross-platform apps)
移动框架：Flutter（跨平台应用）

- Frameworks are like pre-built foundations. You save time from starting from the scratch by adding your customize features to the foundations.

框架如同预先搭建好的地基。你无需从零开始，只需在地基上添加定制化功能，从而节省大量时间。

- How they work: Provide common features (e.g., security, database support) that you customize.

工作原理：提供可自定义的通用功能（例如安全性、数据库支持）。

- Analogy: Think of a framework as a house's basic infrastructure such as plumbing, electricity, roofing and wall. You can design the details and add your own fitting.

类比：把框架想象成房子的基础结构，比如管道、电路、屋顶和墙壁。你可以设计细节并添加自己的装饰。

Extending Framework 扩展框架

- Extension methods:

拓展方法

- **Inheritance:** Create subclasses from framework classes.

继承：从框架类创建子类。

- **Callbacks:** Register functions to be called at specific times or events.

回调函数：在特定时间或事件发生时注册要调用的函数。

- **Hooks:** Insert custom code at predefined points.

钩子：在预定义节点插入自定义代码。

Mini Research

- Imagine you're building a dynamic web app—say, a live music playlist editor—using a framework like React or Django. Your app needs to respond to user actions and lifecycle events. Dive into the roles of **callbacks** and **hooks**: how do they work in a web framework, and what sets them apart in bringing your app to life?

假设你正在使用React或Django这类框架构建一个动态Web应用——比如一个实时音乐播放列表编辑器。你的应用需要对用户操作和生命周期事件作出响应。深入了解回调函数与钩子的作用：它们在Web框架中如何运作？在赋予应用生命力的过程中，它们各自展现出怎样独特的价值？

Challenges of Application Framework

- **Learning Curve:** Steep initial effort to understand and adapt the framework (e.g., mastering Django's structure).

学习曲线：初期需要付出较大努力来理解和适应框架（例如掌握Django的结构）。

- **Overhead:** May include unused features, bloating the app (e.g., extra libraries in React).

开销：可能包含未使用的功能，导致应用臃肿（例如React中的额外库）。

Application Framework vs. Libraries

- **Application Frameworks:**

- Structure for entire apps (e.g., Django).

- Controls flow—you extend it.

- **Software Libraries:**

- Tools for specific tasks (e.g., Requests).
- You control flow—call as needed.

	Application Framework	Software Library
Definition	A pre-built, reusable structure or skeleton that provides a foundation for developing applications. Django (web apps), Spring (enterprise apps)	A collection of reusable functions, classes, or utilities that developers call to perform specific tasks NumPy (math operations), jQuery (DOM manipulation)
Control Flow	Uses an inversion of control (IoC) model — <i>the framework controls the flow</i> . Developers plug their code into predefined entry points (e.g., hooks, callbacks) dictated by the framework.	The developer retains control—you <i>call the library</i> when you need its functionality
Scope and Scale	Broad scope, often covering an entire application domain	Narrower scope, focusing on specific tasks or features
Customization	Customization happens within the framework's constraints—developers extend or override its predefined behaviors	More flexible—developers use only what they need and integrate it however they want, with no imposed architecture.
Dependency	Your application depends heavily on the framework; it's the backbone of the system, and removing it would require a major rewrite.	Dependency is lighter—you can swap out or remove a library with less impact, as it's not integral to the app's structure.

Software Product Line (SPL)

- Building a family of products
构建产品家族
- Definition: A set of related software products sharing a common core but specialized for different needs.
定义：一套共享共同核心但针对不同需求专门设计的相关软件产品。
- Example: Salesforce offer CRM variant for sales, support, and marketing all built from one SPL
Salesforce为销售、支持与市场营销提供统一平台构建的CRM解决方案。
- Benefits:
 - Reuse across multiple products
跨产品复用
 - Faster development for new variants
新变体的开发速度更快
- Analogy: Car models with different features, but using the same engine under the hood
类比：不同配置的汽车型号，但引擎盖下使用的是相同的发动机。
- SPLs are ideal when you need to create similar products with slight variations. The reuse in SPL is at the domain level.
当您需要创建具有细微差异的相似产品时，软件产品线（SPL）是理想选择。SPL中的复用是在领域层面进行的。

SPL Specializations

Type of specialization	Description	Example
Platform specialization	Adapt for different devices	Web, mobile, embedded
Functional specialization	Add features for specific users	Sales, support
Environment specialization	Adjust for different operating environment	High-security, standalone
Process specialization	Support different workflow	Cash payment, credit card, debit card

Challenges for SPL

- **Upfront Investment:** High cost and time to design a reusable core (e.g., defining shared CRM features).
前期投入：设计可复用核心功能（例如定义共享CRM特性）需要高昂的成本和时间。
- **Complexity:** Managing variants can get messy as the family grows (e.g., updating all apps consistently).
复杂性：随着字体家族不断扩充，管理变体可能变得棘手（例如，需要同步更新所有应用程序）。

Exercise

- A software development team is tasked with creating a fitness tracking application that supports multiple user roles (athletes, coaches, health professionals), includes workout planning features, and integrates with external wearable devices (e.g., Fitbit). The client demands delivery within five months and prioritizes maintainability.
- Complete the following structured short-answer questions based on the scenario:
 1. Name **one** software reuse technique suitable for this project.
 2. State **two reasons** why the technique you chose in part 1) is suitable for this project.

Commercial Off-The-Shelf (COTS)

- Ready-made software for quick deployment
即用型软件，快速部署
- Definition: Pre-built software that you can buy and adapt without changing the source code
定义：预构建的软件，您可以在不修改源代码的情况下购买并进行调整。
- Example: WordPress for websites, HubSpot for CRM, SAP for ERP
示例：WordPress用于网站，HubSpot用于CRM，SAP用于ERP

COTS vs. SPL

COTS	SPL
• Single product for broad use • Limited customization • Ready to use	• Family of products for specific domains • Highly customizable • Requires upfront works

Salesforce Case Study

- Internally, Salesforce develop its range of products using SPL strategy
在内部，Salesforce采用SPL策略来开发其产品系列。
- Market its products as individual COTS product to external customers
将产品作为独立的商用现成品向外部客户进行营销
- An SPL could be packaged and sold as COTS products
SPL可打包并作为商用现货产品出售。
- However, SPL is not always sold as COTS products
然而，SPL并不总是作为现成的商业产品出售。

Testing COTS Systems

- Testing of COTS focus on integration
- Key focus: To test how the COTS software interacts with your operational processes
- COTS is pre-tested, but integration with you operation may not smooth
- Example:
 - Ensure the COTS CRM integrates well with your existing email system
 - Ensure the COTS CRM works coherently with your process to manage customers

Challenges for COTS

- **Limited Customization:** Configuration-only approach restricts flexibility (e.g., WordPress can't be fully rewritten).
- **Vendor Dependency:** Reliant on vendor updates or support (e.g., delays in fixing SAP bugs).

Exercise

You are a software engineering consultant hired by a company that develops mobile applications for different industries, such as healthcare, finance, and education. The company is considering adopting software reuse techniques to reduce development time and costs while maintaining high quality. They are particularly interested in understanding the differences between **Application Frameworks** and **Software Product Lines (SPL)**.

你是一名受聘于一家公司的软件工程顾问，该公司为医疗、金融和教育等不同行业开发移动应用。公司正考虑采用软件复用技术，以在保持高质量的同时减少开发时间和成本。他们特别希望了解应用框架和软件产品线之间的区别。

Scenario:

The company plans to develop a new set of mobile apps for small businesses, including:

公司计划为小型企业开发一套新的移动应用程序，包括

- A point-of-sale (POS) app for retail stores.
面向零售商店的销售点（POS）应用程序
- An appointment scheduling app for service-based businesses.
面向服务型企业的预约安排应用
- An inventory management app for warehouses.

一款适用于仓库的库存管理应用。

- All apps need to run on both iOS and Android, and they must share common features like user authentication, payment processing, and data analytics.

所有应用程序都必须在iOS和Android系统上运行，并且必须共享用户身份验证、支付处理和数据分析等共同功能。

- Tasks:
 - Define **Application Frameworks** and **Software Product Lines (SPL)**, highlighting their key differences.
定义应用框架和软件产品线（SPL），并强调它们的关键区别。
 - Recommend which technique (Application Frameworks or SPL) is more suitable for the company's needs. Justify your choice with at least three specific reasons based on the scenario.
推荐哪种技术（应用框架或SPL）更符合公司需求。基于情景至少提出三个具体理由来论证您的选择。
 - Identify two potential challenges the company might face when implementing your recommended technique.
识别公司在实施推荐技术时可能面临的两个潜在挑战。

Discussion

- For each strategy (Application Framework, SPL, and COTS), explain how it tackles *at least two* of the essential difficulties discussed in the essay written by Fred Brooks “No Silver Bullet”, using examples to show your strategy in action.

针对每种策略（应用框架、SPL和COTS），请结合弗雷德·布鲁克斯在《没有银弹》一文中讨论的至少两个本质性难题，说明该策略如何应对这些挑战，并通过实例展示策略的实际应用。

- Your task is to challenges the below given answer.

你的任务是对以下给出的答案提出质疑。

Answer:

- **Application Frameworks:**
 - **Complexity:** Frameworks like Django reduce complexity by providing a structured blueprint—pre-built modules for carts, payments, and user accounts simplify the app's design. Instead of juggling chaotic code, you follow a clear MVC pattern.
复杂度：诸如Django这类框架通过提供结构化蓝图来降低复杂度——预构建的购物车、支付和用户账户模块简化了应用设计。您无需再处理混乱的代码，只需遵循清晰的MVC模式即可。
 - **Changeability:** They handle evolving needs well; adding a new payment gateway (e.g., Stripe) is just a matter of plugging into Django's extensible architecture via middleware or apps.
可变更性：它们能很好地应对不断变化的需求；添加新的支付网关（例如Stripe）只需通过中间件或应用接入Django的可扩展架构即可。
- **Software Product Lines (SPL):**
 - **Complexity:** SPLs tame complexity by creating a reusable core (e.g., shared checkout and inventory logic) for variants like mobile and desktop e-commerce apps, reducing redundant design work.
复杂性：软件产品线通过为移动端和桌面端电商应用等变体创建可复用的核心（例如共享的结算和库存逻辑），来驯服复杂性，减少冗余的设计工作。

- **Conformity:** They align with real-world constraints by specializing the core—e.g., a mobile version conforms to touch inputs, while desktop supports keyboard navigation, all from one base.

一致性：它们通过核心专业化来适应现实世界的约束——例如，移动版本适配触控输入，而桌面端支持键盘导航，这一切都基于同一基础架构。

- **COTS:**

- **Complexity:** COTS like Shopify slashes complexity—you get a pre-built e-commerce solution with carts and analytics out of the box, no need to design from scratch.

复杂性：像Shopify这样的商业现成解决方案大幅降低了复杂度——您获得的是开箱即用的预构建电商平台，包含购物车和分析功能，无需从零开始设计。

- **Changeability:** It adapts to changes via configuration (e.g., adding a discount feature through plugins), though major shifts might hit customization limits.

可变更性：系统通过配置适应变化（例如通过插件添加折扣功能），但重大变更可能触及定制化限制。

Reuse in the Real World

- Overview: Netflix reuses micro-services across its platform

概述：Netflix在其平台范围内复用微服务架构

- Benefit:

- Scalability: Reuse services enable Netflix to

- 可扩展性：服务复用机制使Netflix能够

- Handle million of users by adding more micro-service instances to a specific Netflix service
通过为特定Netflix服务增加更多微服务实例来应对数百万用户。

- Easily add new Netflix service using existing micro-services
轻松使用现有微服务添加新的Netflix服务

- Reliability: Tested micro-services can be reused without testing again

可靠性：经过测试的微服务无需再次测试即可重复使用

Reuse in the Real World

- Scalability means growing without breaking

可扩展性意味着在不崩溃的前提下持续发展

- Reuse micro-services enable Netflix to add more service instances and servers without disrupting users

复用微服务使得Netflix能够在不影响用户的情况下增加更多服务实例和服务器。

- Flexibility: Deploy and scale individual services (e.g., adding more streaming instances during peak times) without affecting others.

灵活性：部署和扩展单个服务（例如，在高峰时段增加更多流媒体实例）而不会影响其他服务。

Why Reuse Matters

- Summary
 - Reuse saves time, money, and effort.
节省时间、经济、努力
 - Techniques like frameworks, SPL, and COTS offer different ways to reuse.
框架、SPL和COTS等技术提供了不同的复用途径。
 - Real-world companies rely on reuse to stay competitive.
现实世界中的公司依赖重复利用来保持竞争力。
- As software engineers, mastering reuse will make you more efficient and valuable.
作为软件工程师，掌握复用技术将使您的工作效率更高、价值更大。
- How can you apply reuse in their own projects?
如何在自己的项目中应用复用策略？